Hardware Requirements

- GPU: RTX 5060 8GB (or equivalent)
- RAM: 16GB+
- Disk: A few GB for LanceDB + graph files

Software Prerequisites

```
# Python 3.10+
pip install fastembed==0.6.0 numpy lancedb pandas datasets ragas
langchain requests

# Ollama for local LLMs (https://ollama.com)
ollama pull qwen2.5:7b-instruct    # Generator + Answerer (~4.5 GB
VRAM)
ollama pull llama3.1:8b            # Question Critic (different
architecture)
ollama pull nomic-embed-text:latest # RAGAS document clustering (~0.3
GB)
```

API Keys (for evaluation judges only)

```
export GROQ_API_KEY_1="gsk_..."           # Account 1 (1000 RPD)
export GROQ_API_KEY_2="gsk_..."           # Account 2 (1000 RPD)
export GROQ_API_KEY_3="gsk_..."           # Account 3 (1000 RPD)
export GROQ_API_KEY="$GROQ_API_KEY_1"     # Start with account 1

export GEMINI_API_KEY="AIzaSy..."         # Google AI Studio (1500 RPD)
export OPENROUTER_API_KEY="sk-or-v1-..."  # DeepSeek R1 free tier
```

Data Requirements

You need a `cognee_system/` folder with:

- `last_results.json` (knowledge graph: nodes + edges)
- `cognee.lancedb/` (vector database built with MiniLM 384-dim)

Phase 1: Question Generation (`generate_questions.py`)

What it does: Creates test questions automatically from your documents.

How it works:

1. Load documents from LanceDB
2. Use RAGAS with an LLM to generate 3 types:
    - Simple (20%): "What is X?" — straightforward facts
    - Reasoning (45%): "Why does X cause Y?" — needs thinking
    - Multi-context (35%): Needs info from 2+ different documents
3. Quality filter (QGE): A judge LLM checks each question for answerability, clarity, specificity, ground truth correctness, and duplicates

Models used:

```
# Generator: Qwen (multilingual, 128K context)
generator_llm = Ollama(model="qwen2.5:7b-instruct", temperature=0.7)

# Critic: Llama (different architecture to eliminate shared-bias)
critic_llm = Ollama(model="llama3.1:8b", temperature=0.0)

# Document clustering: Nomic (independent from retrieval embedding)
embeddings = OllamaEmbeddings(model="nomic-embed-text:latest")
```

Run:

```
export QGE_JUDGES="llama-3.3-70b-versatile,gemini-2.0-flash"

python generate_questions.py \
  --dataset outline_of_science \
  --n 1000 \
  --model qwen2.5:7b-instruct
```

Output:

- `ragas_questions_outline_of_science.json` (accepted questions)
- `ragas_questions_outline_of_science_qge_report.json` (quality stats)

Phase 2: Retrieval (`data_provider.py`)

What it does: Finds the most relevant information for each question.

Steps:

1. Query Analysis: Detects question type (reasoning/entity/general)
2. Vector Search: Converts question to 384-dim embedding (MiniLM), searches LanceDB by cosine similarity
3. Graph Search: Searches nodes/edges with embedding similarity + keyword boost + neighbour expansion
4. Reasoning Chains: BFS multi-hop path detection for "why/how" questions (up to 3 hops)
5. Fusion: Combines vector + graph + chains, deduplicates, ranks, keeps top 8

Key code:

```
ctx = await get_rag_context("What is electricity?",
"outline_of_science")
# Returns: contexts, graph_insights, reasoning_chains, stats
```

Phase 3: Answer Generation (`generate.py` + `llm.py` + `prompt.py`)

What it does: Produces grounded answers from retrieved context.

The Prompt Builder builds structured prompts with relevance labels and graph insights.

The LLM Router tries fallback chain: Ollama → HuggingFace → Groq → Gemini → OpenRouter → Refusal

Critical for evaluation: Lock to single model for fair scores:

```
export EVAL_MODE=1                    # Force single model
export LLM_MODEL="qwen2.5:7b-instruct" # Unified model
export ALLOW_API_FALLBACK=0           # No provider mixing
```

The Generator:

- Calls LLM with prompt
- Grounding check (diagnostic only)
- Extracts citations [1], [2]
- Returns structured result with model, language, sources

Phase 4: Evaluation (`score_ragas.py`)

What it does: Scores every answer on 4 metrics using LLM-as-judge.

The 4 Metrics:

Table

Metric	Meaning	Good Score
Faithfulness	Does answer stick to context?	> 0.7
Answer Relevancy	Does it answer the question?	> 0.7
Context Precision	Is retrieved context relevant?	> 0.7
Context Recall	Did retrieval find needed info?	> 0.7

Tiered Evaluation:

- Tier 1: One fast judge (`llama-3.3-70b-versatile` via Groq) scores everything
- Tier 2: Only suspicious cases get 3-judge debate:
  - Low scores (avg < 0.6 or faithfulness < 0.4)
  - Random audit samples (10% of good ones)

Debate Panel:

- Judge A: `llama-3.3-70b-versatile` (Groq) — continuity with Tier 1

- Judge B: `gemini-2.0-flash` (Google) — architectural diversity
- Judge C: `deepseek/deepseek-r1:free` (OpenRouter) — chain-of-thought reasoning
- Moderator: `gemini-2.0-flash` — resolves disagreements

High-consensus shortcut: If σ < 0.08, bypass moderator.

Retrieval Metrics: Compares retrieved chunks against gold contexts:

- MRR, Recall@K, NDCG

How to Run (Step-by-Step)

Step 1: Generate Questions (~30 min, all local)

```
export QGE_JUDGES="llama-3.3-70b-versatile,gemini-2.0-flash"

python generate_questions.py \
  --dataset outline_of_science \
  --n 1000 \
  --model qwen2.5:7b-instruct
```

Step 2: Run Retrieval + Generation (~3-5 hours, all local)

```
# CRITICAL: Lock to one model for fair evaluation
export EVAL_MODE=1
export LLM_MODEL="qwen2.5:7b-instruct"
export ALLOW_API_FALLBACK=0

python run_pipeline_on_ragas_questions.py \
  --dataset outline_of_science \
  --questions ragas_questions_outline_of_science.json
```

Step 3: Score Results (~10-12 hours with 3 Groq accounts)

```
# Switch to scoring environment
unset EVAL_MODE
export ALLOW_API_FALLBACK=1

export RAGAS_JUDGE_MODEL="llama-3.3-70b-versatile"
```

```
export
TIER2_DEBATE_JUDGES="llama-3.3-70b-versatile,gemini-2.0-flash,deepseek/
deepseek-r1:free"
export MODERATOR_MODEL="gemini-2.0-flash"

python score_ragas.py \
  --input ragas_pipeline_results_outline_of_science.json \
  --questions ragas_questions_outline_of_science.json \
  --dataset outline_of_science
```

When Groq hits RPD limit (429 errors):

```
# Rotate to next account
export GROQ_API_KEY="$GROQ_API_KEY_2"  # or _3
# Script auto-resumes from checkpoint
```

Model Map Summary

Component	Model	Type	Why
Question Generator	`qwen2.5:7b-instruct`	Local Ollama	Multilingual, 128K context
Question Critic	`llama3.1:8b`	Local Ollama	Different architecture, eliminates shared-bias
RAGAS Clustering	`nomic-embed-text:latest`	Local Ollama	Document clustering (independent from retrieval)
Retrieval Embedding	`MiniLM-L12-v2` (384-dim)	fastembed	Matches existing index

Answer Generator	`qwen2.5:7b-instruct`	Local Ollama	Unified in eval mode, deterministic
Tier 1 Judge	`llama-3.3-70b-versatile`	Groq API	Strongest evaluator, 30 RPM
Tier 2 Judge A	`llama-3.3-70b-versatile`	Groq API	Continuity with Tier 1
Tier 2 Judge B	`gemini-2.0-flash`	Google API	Free, diverse architecture
Tier 2 Judge C	`deepseek-r1:free`	OpenRouter	Free, chain-of-thought
Moderator	`gemini-2.0-flash`	Google API	Fast, resolves disagreements

Key Design Decisions

Table

Decision	Rationale
qwen7B instead of qwen14B	14B requires 8.7GB, exceeds RTX 5060 8GB → CPU offload introduces timing noise

llama3.1:8b critic instead of qwen7B	Different model family eliminates shared blind spots
MiniLM for retrieval	Matches existing 384-dim index; changing requires full re-ingestion
nomic for RAGAS clustering	Better document clusters than MiniLM, independent task
3 Groq accounts	Free tier = 1000 RPD/account; 1000 questions + Tier 2 = ~1200 calls
Phase separation	`ALLOW_API_FALLBACK=0` during generation ensures single-model reproducibility

---

Output Files

Table

File	Contents
`ragas_scores_detailed.json`	Every question with all 4 scores
`weak_cases.json`	Questions with faithfulness < 0.5
`retrieval_metrics.json`	MRR, Recall@K, NDCG

`tier1_scores.json`	First-pass scores
`tier2_debate_results.json`	Multi-judge debate details
`tiered_final_scores.json`	Final merged scores
`tiered_training_labels.json`	High/medium/rejected labels

Expected Results

```
Overall Scores:
  faithfulness        : 0.712   [▰▰▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱]
  answer_relevancy    : 0.685   [▰▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱]
  context_precision   : 0.743   [▰▰▰▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱]
  context_recall      : 0.691   [▰▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱]
  Overall average     : 0.708

Retrieval Metrics:
  mrr         : 0.5423
  recall@1    : 0.4100
  recall@3    : 0.6733
  recall@5    : 0.8100
  ndcg@5      : 0.7234
```

Troubleshooting

Problem	Solution

Groq 429 (RPD limit)	Rotate `GROQ_API_KEY` to next account
Empty LLM response	Check Ollama running: `ollama ps`
Zero retrieval	Verify LanceDB path, graph has nodes
Embedding mismatch	Ensure query model matches index dimension (384)
OOM during generation	Ollama unloads models sequentially; peak ~5GB

MRR (Mean Reciprocal Rank)

What it measures: How high is the first correct answer in your results?

Recall@K

What it measures: Did the correct answer appear in the top K results?

NDCG (Normalized Discounted Cumulative Gain)

What it measures: Is the ranking order good?

Metric	Your Score	"Good" Score	Why It's "Low"
MRR	0.54	>0.7	First correct result is often at position 2+, not #1
Recall@1	0.41	>0.6	Only 41% of the time is the top result correct
Recall@5	0.81	>0.9	Even with 5 results, 19% of answers are missing

This is actually NORMAL for GraphRAG because:

1. GraphRAG retrieves multiple types: vectors + graph nodes + reasoning chains — some are bound to be less relevant
2. Your fusion strategy prioritizes diversity over precision — you want different perspectives, not just one perfect chunk
3. The "gold" context is subjective — RAGAS generated the question from specific chunks, but other chunks might also contain the answer

---

The Real Test: Do Low Retrieval Scores Hurt Answer Quality?

Scenario	Retrieval	Answer Quality	Fix
Recall@5 = 0.8, Faithfulness = 0.7	Okay	Good	None needed
Recall@5 = 0.8, Faithfulness = 0.3	Okay	Bad	LLM is hallucinating, not retrieval
Recall@5 = 0.3, Faithfulness = 0.3	Bad	Bad	Retrieval is failing

Your expected scores (faithfulness ~0.71, recall@5 ~0.81): Retrieval is good enough—the answer quality bottleneck is likely the LLM's ability to synthesize, not finding the right chunks.

How to Improve Retrieval (If Needed)

Approach	Impact	Effort
Increase `MAX_CONTEXT` from 8 to 12	Higher recall	1 line change
Lower `CUSTOM_RELEVANCE_THRESHOLD` from 0.4 to 0.3	More chunks retrieved	1 line change

Tune `vector_k` and `graph_k` per query type	Better balance	Already adaptive
Upgrade to `multilingual-e5-large`	+5-10% MTEB	Requires re-ingestion
Add reranking (ColBERT)	+15% MRR	Complex

➔ *Retrieval achieves MRR=0.54 and Recall@5=0.81, indicating the system consistently locates relevant information within the top 5 results. The sub-perfect scores reflect our intentional diversity-focused fusion strategy, which prioritizes multi-source coverage over single-source precision — a trade-off justified by strong downstream faithfulness scores (0.71).*